

23. Claude Token Saving

Frontmatter

video_number: 23
folder_title: Claude_Token_Saving
series: Building an AI Business From Scratch
video_type: Systems / Optimization
hook_type: Problem hook -- hitting limits
framework_phrase: Less context, more focus
script_versions: 1
status: ideation

Reference Videos

- Reduce Claude Code Token usage by 90% -- <https://www.youtube.com/watch?v=FzgLeJPChJc>
- Claude Code - 10 Tricks to Optimise Token Usage -- <https://www.youtube.com/watch?v=XB6iJvrLFmE>
- Reduce tokens by 90% in Claude Code | 9 techniques that work -- <https://www.youtube.com/watch?v=0MPDOn9YWik>

The One Thing

Token waste in Claude Code is mostly a context problem you can fix -- not a rate limit problem you have to accept.

Hook

Claude's hitting your daily limit before noon. Here are 9 things I changed that cut my usage by 90% without slowing down what I build.

Target Viewer

Claude Code power users hitting rate limits mid-session or getting surprised by large API bills, who want tactical fixes they can apply today.

Payoff

After watching, the viewer can identify the 3 biggest token drains in their Claude Code setup and apply at least 3 of the 9 techniques immediately -- with a rough sense of the savings each one produces.

Beat Structure

Beat -- 1 -- Why Tokens Drain Fast

Point: Context window accumulation is the main driver -- Claude reads everything in the conversation on every turn, so long sessions compound fast.

Visual: Diagram: context window filling up turn by turn, cost meter rising

Target words: context window, accumulation, session cost

Beat -- 2 -- CLAUDE.md Trimming

Point: A bloated CLAUDE.md is read on every request -- cutting it from 2,000 to 400 tokens saves roughly 1,600 tokens per turn, which compounds across hundreds of turns per day.

Visual: Before/after CLAUDE.md size comparison, token counter

Target words: CLAUDE.md, system prompt, per-request cost

Beat -- 3 -- Selective File Reads

Point: Asking Claude to 'understand the codebase' dumps every file into context -- instead, name the specific files needed for each task.

Visual: Screen recording: vague prompt with massive file load vs specific prompt with 2 files

Target words: file read, context load, specific prompts

Beat -- 4 -- /compact and Fresh Sessions

Point: /compact summarizes the current conversation into fewer tokens; starting a new session after a logical task boundary avoids carrying dead context forward.

Visual: Terminal: /compact command, before/after token count display

Target words: /compact, session boundary, context reset

Beat -- 5 -- Subagents for Isolation

Point: Spawning a subagent for a contained task runs it in a fresh context -- the subagent completes the task and returns only its output, keeping the main session clean.

Visual: Diagram: main session + spawned subagent with isolated context, result returned

Target words: subagent, context isolation, agent spawn

Beat -- 6 -- Specific vs Vague Prompts

Point: Vague prompts trigger exploratory behavior -- Claude searches more files and generates more reasoning tokens before answering. Specific prompts get specific answers.

Visual: Side-by-side: vague prompt token count vs specific prompt token count for same task

Target words: prompt specificity, reasoning tokens, output quality

Analogy Bank

- Context accumulation: like a whiteboard that never gets erased -- by hour 3 you're reading around old drawings to find space for new ones. [concept: why sessions get expensive]
- CLAUDE.md trimming: like cutting a meeting agenda from 20 items to 5 -- the meeting runs faster and everyone reads the whole thing. [concept: system prompt size]

Thesis Line Location

Beat 1 -- the core insight is that tokens drain from context accumulation, not from Claude being slow or dumb -- once you understand that, all 9 techniques make intuitive sense.

Framework Phrase

Less context, more focus

Specificity Bank

- CLAUDE.md: trimming from 2,000 to 400 tokens saves ~1,600 tokens per turn
- /compact command in Claude Code terminal
- Subagent spawn via Agent tool in Claude Code
- 90% reduction reported across multiple independent builders (3 YouTube videos confirm)
- Claude Code reads the full context window on every turn -- no partial reads

Constraints

- Do not promise exact savings figures -- vary by workflow; say 'up to 90%' with caveats
- Do not cover API-level prompt caching here -- save that for a dedicated video
- Focus on Claude Code specifically, not the API or other tools

CTA Direction

Next video: Codex has a completely different billing model -- here's how token saving works differently over there, and which approach actually costs less.

Personal Angle

[DANIEL TO FILL] -- e.g. the session where you hit your limit at 10am and had to stop working, what that forced you to figure out

Vulnerability Moment

[DANIEL TO FILL] -- e.g. the API bill that shocked you before you figured out it was your CLAUDE.md doing 80% of the damage